Stock Market Return Distribution Analyzer

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

By

Srikar J	**: 2420030111**
G Venkata Varun	**: 2420030187**
K Pranav	**: 2420030228**

Under the Esteemed Guidance of

B Prathusha

Department of Computer Science and Engineering

AY- 2026-2027

Semester: 5

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



Declaration

The Project Report entitled "**Stock Market Return Distribution Analyzer**" is a record of Bonafide work of **Srikar J – 2420030111, G Venkata Varun – 2420030187, K Pranav – 2420030228** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Srikar J – 2420030111

G Venkata Varun – 2420030187

K Pranav – 2420030228

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is certify that the mini project based report entitled "**Stock Market Return Distribution Analyzer**" is a bonafide work done and submitted by **Srikar J – 2420030111, G Venkata Varun – 2420030187, K Pranav – 2420030228** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Engineering, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr Nalajala Sunanda**, and my guide **B Prathusha,** Department of Computer Science and Engineering which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Venkateshwara Rao Pulapati**, Head of the ***Department for Computer Science and Engineering*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

Srikar J – 2420030111

G Venkata Varun – 2420030187

K Pranav – 2420030228

Abstract

The Stock Market Return Distribution Analyzer is a financial analysis system designed to study the behavior and distribution of stock market returns over a selected period. The system uses historical stock price data to calculate daily, weekly, or monthly returns and organizes the data for further statistical analysis. It provides a simple and efficient way to understand how stock prices change over time and how these changes affect investment returns.

The system analyzes important statistical measures such as mean, median, variance, standard deviation, skewness, and kurtosis. These measures help identify the average return, level of risk, volatility, and overall shape of the return distribution. Graphical representations such as histograms and distribution curves can also be used to identify patterns, extreme values, and unusual market movements.

The main objective of this project is to provide a better understanding of the relationship between risk and return in the stock market. By analyzing historical return distributions, investors and researchers can compare different stocks, assess their volatility, and support more informed investment decisions. The analyzer can therefore serve as a useful tool for financial data analysis, risk assessment, and understanding stock market behavior.

The system is implemented as a local research workspace named Folio, using a React and TypeScript front end and a FastAPI engine built on pandas, NumPy and SciPy. It retrieves auto-adjusted historical data from Yahoo Finance and provides ten analytical views, including distribution analysis, risk analytics, multi-asset comparison and extreme-event detection, together with downloadable research reports.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience	1-3
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	4-5
3	Technology Stack	3.1 Front-end (React, TypeScript) 3.2 Back-end (FastAPI, pandas, NumPy, SciPy) 3.3 Data storage (no database; six-hour cache) 3.4 Project scripts and packaging 3.5 APIs or external services (Yahoo Finance)	6-7
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer (front end, back end, data source) 4.3 Deployment architecture (local, loopback)	8-9
5	Design	5.1 User interface design (screenshots) 5.2 Data flow diagram and data schema (no ER diagram – no database)	10-12
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic (UI rendering, shared state) 6.3 Backend logic (returns, statistics, risk) 6.4 Data connectivity and caching 6.5 API integration 6.6 Security and privacy considerations	13-15
7	Features	7.1 List of main features 7.2 How each feature works (user perspective + technical description)	16-17
8	Testing	8.1 Statistical (unit) testing 8.2 Build and integration verification 8.3 Tools used for testing 8.4 Test cases and results	18-19
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of frontend and backend	20-21
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations or known bugs	22-23
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations or optimizations	24

12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development	25-26
13	References	Books, tutorials, APIs, documentation sites used	27

CHAPTER 1

Introduction

1.1 Background of the project

Stock prices change every trading day, but the price level alone says little about how risky or rewarding an investment has been. What matters to an analyst is the **return** – the percentage change from one observation to the next – and, more importantly, the **distribution** of those returns: where they cluster, how widely they spread, how lopsided they are and how often extreme moves occur.

The **Stock Market Return Distribution Analyzer**, implemented as a research workspace named **Folio**, studies the behaviour of stock market returns over a selected period. It downloads historical adjusted price data from Yahoo Finance, converts it into daily, weekly or monthly returns and applies descriptive statistics and visual analysis to explain how the returns behave. The system is built with a React and TypeScript front end and a FastAPI engine that uses pandas, NumPy and SciPy for all calculations.

1.2 Problem statement

Investors, students and researchers usually look at price charts, which hide the statistical character of an asset. Questions such as “Is the average return positive?”, “How volatile is this stock compared with another?”, “Are the returns symmetric?” and “Are extreme days more frequent than a Normal distribution would predict?” cannot be answered by looking at a price line alone. Performing this analysis manually in spreadsheets is slow, error-prone and difficult to repeat for different assets, periods and return horizons.

There is therefore a need for a single, easy-to-use tool that fetches clean historical data, calculates returns correctly, reports the standard statistical measures and presents the results visually so that the risk–return behaviour of stocks can be understood and compared quickly.

1.3 Scope of the project

The scope of the project covers descriptive, historical analysis of market data. It includes:

- Historical adjusted prices from Yahoo Finance for stocks, indices and other symbols, with a searchable asset palette and support for custom Yahoo symbols.
- Daily, weekly (Friday-ending) and monthly return calculation over a user-selected date range.
- Six sample statistics – mean, median, variance, standard deviation, skewness and excess kurtosis – together with histograms, kernel density estimates, Normal fits, Q–Q plots, violin, box and ECDF plots.
- Risk analytics (rolling volatility, drawdown), comparison of two to five assets, multi-horizon analysis and extreme-event detection.
- Calculated research insights, a downloadable PDF report, a presentation mode and CSV export of the data.

The project deliberately does **not** implement trading, price prediction, order placement or buy/sell recommendations, and it does not infer the causes of historical events. All findings are descriptive and derived from the calculated returns.

1.4 Objective(s)

The main objective of the project is to provide a better understanding of the relationship between risk and return in the stock market by analysing historical return distributions. The specific objectives are:

- To retrieve and clean historical stock price data and compute daily, weekly and monthly returns.
- To calculate the mean, median, variance, standard deviation, skewness and kurtosis of the return series.
- To visualise return distributions using histograms and distribution curves so that patterns, extreme values and unusual market movements can be identified.

- To compare different stocks in terms of return, volatility and distribution shape.
- To support more informed, data-driven interpretation of risk through an interactive and reproducible workspace.

1.5 Importance of the application

Many financial models assume that returns follow a Normal distribution, but real market returns usually have heavier tails and some asymmetry. By showing the observed distribution against a fitted Normal curve, and by reporting skewness and excess kurtosis, the application makes this difference visible. For example, in the sample analysis shown in Chapter 5 the excess kurtosis is 11.94 against a Normal benchmark of 0, which immediately signals that extreme movements are far more frequent than a Normal model would suggest.

The application also removes the repetitive manual work of data collection, cleaning and calculation, uses a documented statistical methodology (sample estimators with ddof = 1, bias-corrected skewness and kurtosis) and can be run entirely on a local computer without any account or subscription.

1.6 Target users/audience

- **Students and academic researchers** studying data science, statistics or financial markets.
- **Individual investors and analysts** who want to understand the volatility and distribution of the assets they follow.
- **Educators** who need an interactive tool, with a presentation mode, to explain concepts such as skewness, kurtosis and fat tails.
- **Risk and data-analysis practitioners** who need quick, reproducible descriptive statistics for a chosen asset and period.

CHAPTER 2

System Requirements

2.1 Hardware requirements

Folio runs entirely on the user's own computer, so the hardware requirements are modest. The following configuration is recommended for smooth interaction with the interactive charts.

Component	Recommended specification
Processor	Dual-core 64-bit processor (Intel Core i3 / AMD Ryzen 3 or better)
Memory (RAM)	8 GB recommended (4 GB minimum)
Storage	At least 2 GB free space for the project, Node.js packages and the Python environment
Display	1366 × 768 or higher; a full-HD display is recommended for the dashboard layout
Network	Internet connection for downloading historical data from Yahoo Finance

2.2 Software requirements

Software	Requirement
Operating system	Windows (the supplied launcher scripts are Windows .cmd and PowerShell files)
Python	Python 3.11 or later, with a virtual environment
Node.js	Node.js 20 or later with npm
Python libraries	Packages listed in backend/requirements.txt (FastAPI, pandas, NumPy, SciPy, yfinance)
Front-end packages	Installed with npm ci from the project's lock file
Web browser	A modern web browser (for example Chrome or Edge)

The project also contains an optional **backend/.packages** directory, which is a project-local installation of yfinance. The backend start-up file (run.py) discovers it automatically, so the global Python packages do not have to be modified.

2.3 Development tools and frameworks

- **Languages:** TypeScript (front end) and Python (analytics engine).
- **Front-end framework and styling:** React, Tailwind CSS and a custom editorial design system; Radix UI primitives for accessible dialogs; locally bundled fonts.
- **Charting:** TradingView Lightweight Charts, Plotly.js and Apache ECharts.
- **Back-end framework and libraries:** FastAPI, pandas, NumPy, SciPy and yfinance.
- **Utilities:** npm for building and running the front end, and PowerShell/CMD scripts (start.ps1, stop.ps1, Start Folio.cmd, Stop Folio.cmd) for launching and stopping both services.

CHAPTER 3

Technology Stack

3.1 Front-end (React, TypeScript)

The user interface is a single-page application written in **React** and **TypeScript**. TypeScript adds static typing, which is valuable in an application that passes many numerical series between components. Styling is done with **Tailwind CSS** together with a custom editorial design system that provides a warm light theme and a charcoal dark theme. Dialogs and overlays use **Radix UI**, the accessibility-focused primitive library that also underlies shadcn/ui. All fonts are bundled locally, so the interface does not depend on a font CDN.

Three charting libraries are used, each for what it does best:

- **TradingView Lightweight Charts** – price charts (line, area and candlestick) with zoom, pan and OHLC inspection; the library's attribution is retained.
- **Plotly.js** – statistical plots such as histograms, density overlays, Q–Q plots, violin and box plots.
- **Apache ECharts** – explorer panels such as heatmaps, timelines and the synchronised crosshair views.

3.2 Back-end (FastAPI, pandas, NumPy, SciPy)

The analytics engine is a **FastAPI** application written in Python. It is responsible for fetching and cleaning data, resampling prices to the chosen horizon, calculating returns and computing every statistic that the interface displays.

- **pandas** – tabular OHLCV data, date handling, cleaning, weekly and monthly aggregation and alignment of several assets on common dates.
- **NumPy** – numerical operations on the return series.

- **SciPy (scipy.stats)** – sample skewness and kurtosis, Gaussian kernel density estimation, the Normal distribution for fitted curves and tail probabilities.

3.3 Data storage (no database)

The project does not use a database server. Historical data are downloaded on demand from Yahoo Finance and held in a **six-hour response cache**; the Refresh button requests a fresh download. Because the analysis is read-only and single-user, a database is not required, and the data schema is described in Section 5.2 instead of an ER diagram.

3.4 Project scripts and packaging

To make the project easy to run on a personal computer, it ships with launcher scripts. **Start Folio.cmd** starts the backend and frontend services and opens the application once both are ready; **Stop Folio.cmd** stops them. The same actions are available through start.ps1 and stop.ps1. The launcher reuses an already running Folio instance and checks readiness before opening the browser.

3.5 APIs or external services

The only external service is **Yahoo Finance**, accessed through the **yfinance** library. The OHLC (open, high, low, close) values are auto-adjusted for corporate actions such as splits and dividends, so the data are historical adjusted values rather than live quotes. The retrieval time and the data source are displayed in the interface. An optional, clearly labelled **synthetic demo dataset** can be switched on for offline demonstration; it is deterministic, opt-in and is never presented as Yahoo market data.

CHAPTER 4

System Architecture

4.1 High-level architecture diagram

Folio follows a three-layer architecture: a presentation layer in the browser, an application/analytics layer implemented as a FastAPI service, and an external data-source layer. Figure 4.1 shows the layers and the way data move between them.

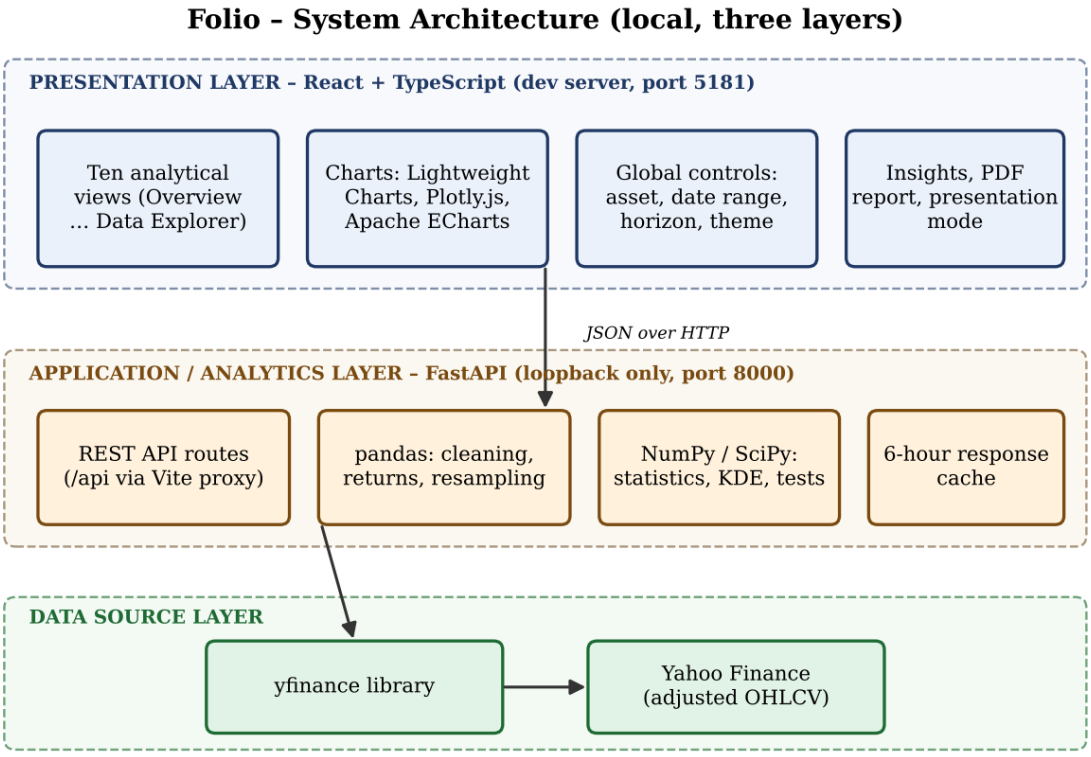


Fig No. 4.1 High-level architecture of Folio

4.2 Description of each layer

Presentation layer (front end). The React application contains the ten analytical views, the global controls (asset, date range and return horizon), the charts, the Insights page with its PDF report and presentation mode, and the Data Explorer. It never calculates statistics itself; it requests prepared results

from the API and renders them. The front end is served on port 5181 and talks to the backend through a local **/api** proxy.

Application and analytics layer (back end). The FastAPI service listens only on the loopback address (port 8000). It receives a request describing the asset, date range and horizon, obtains the cleaned price data, computes the returns and the requested statistics with pandas, NumPy and SciPy, and returns the results as JSON. A six-hour cache prevents repeated downloads of the same data.

Data-source layer. The yfinance library downloads auto-adjusted OHLCV history from Yahoo Finance. The backend cleans the data before use: invalid OHLC rows and duplicate dates are removed, missing volume is filled with zero, and weekends and holidays are not treated as missing quotes.

4.3 Deployment architecture

The system is designed for local deployment on a single computer. There is no cloud hosting or CI/CD pipeline. Both services are started together by the launcher script; the API is bound only to the loopback interface so it cannot be reached from other machines, and the browser reaches it through the front end's proxy at http://127.0.0.1:5181. The only outbound connection is the request from the backend to Yahoo Finance.

CHAPTER 5

Design

5.1 User interface design (screenshots)

The interface uses a quiet, editorial style: a left navigation sidebar with the ten views, a scrolling ticker of sample assets at the top, and a control bar for the asset, date range, horizon (Daily, Weekly, Monthly) and data source. Figure 5.1 shows the **Overview** page for the NIFTY 50 index (^NSEI) between 06-11-2024 and 06-11-2026. It contains the adjusted price chart with line, area and candlestick modes, range buttons (3M, 6M, 1Y, 2Y, 5Y) and a "statistical lens" panel with the mean daily return, volatility, number of observations and a small return-profile histogram.

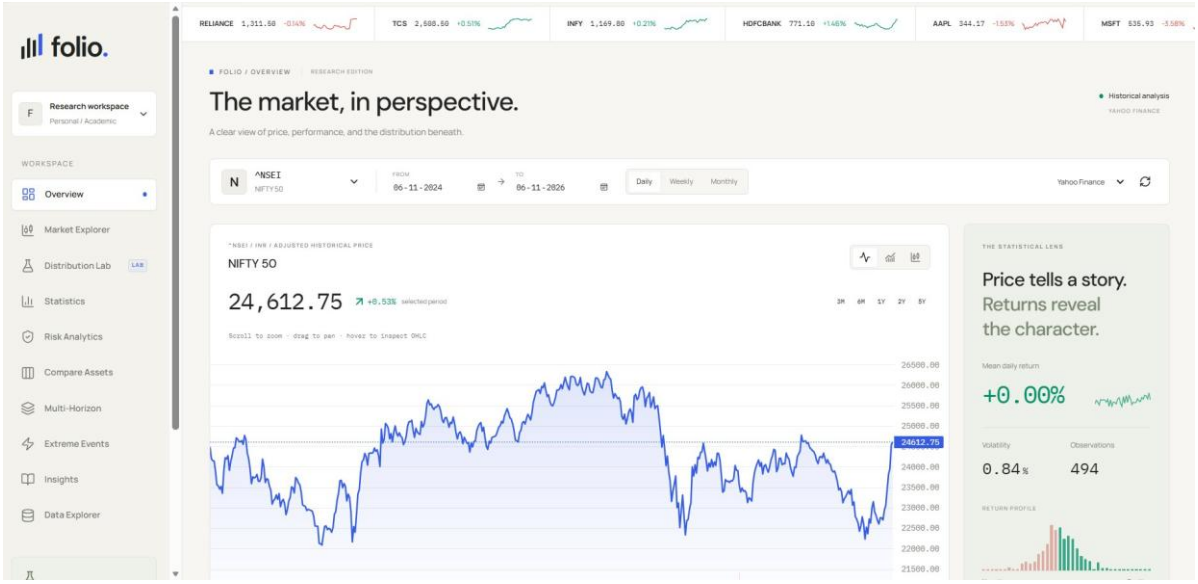


Fig No. 5.1 Overview page – price chart and statistical lens (NIFTY 50)

Figure 5.2 shows the statistics and distribution section of the Overview for a sample of 500 daily observations. The six statistic cards report the mean, median, standard deviation, variance, skewness and excess kurtosis, each with a benchmark note. Below them, the return distribution chart overlays the observed histogram, the kernel density estimate and the fitted Normal curve, and the "balance of returns" card shows the share of positive intervals together with

the best and worst interval. A calendar heat-map of daily returns completes the page.

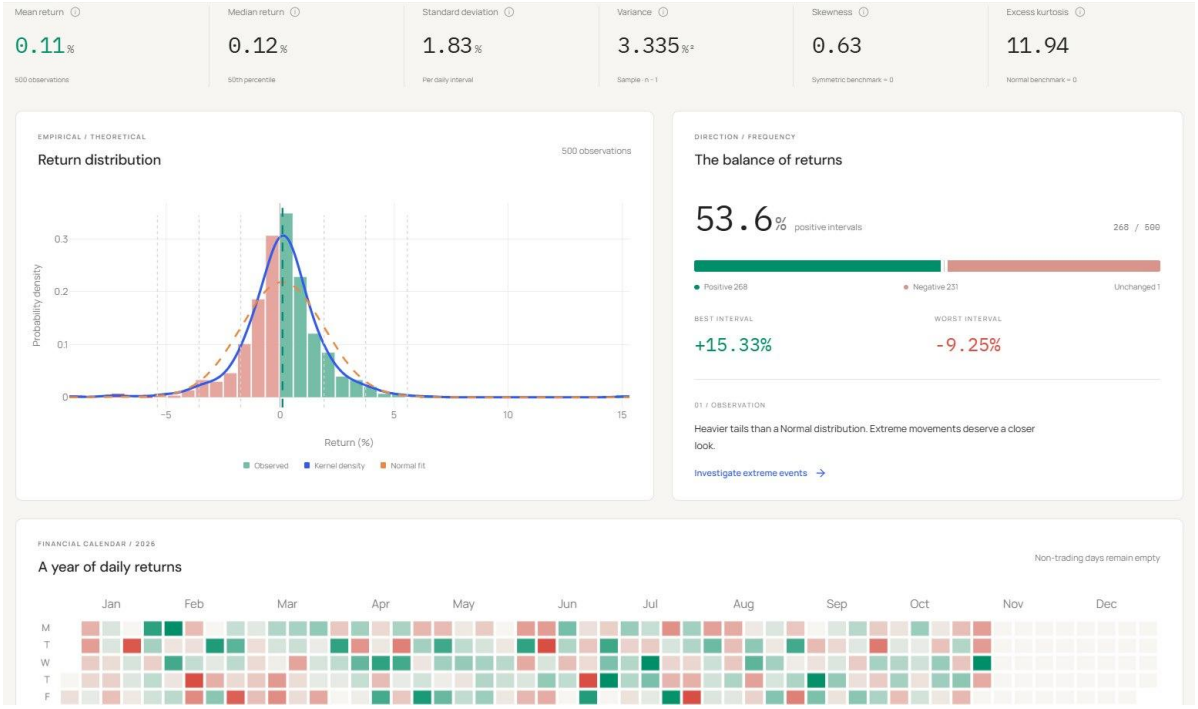


Fig No. 5.2 Statistics cards, return distribution, balance of returns and calendar

5.2 Data flow diagram and data schema

Because the application has no database, the design focuses on how data flow through the system. Figure 5.3 shows the data flow: the user selects an asset, date range and horizon; the backend fetches data from Yahoo Finance and stores the response in the cache; the data are cleaned and converted to returns; statistics are calculated; and the results are returned as charts, insights and the PDF report.

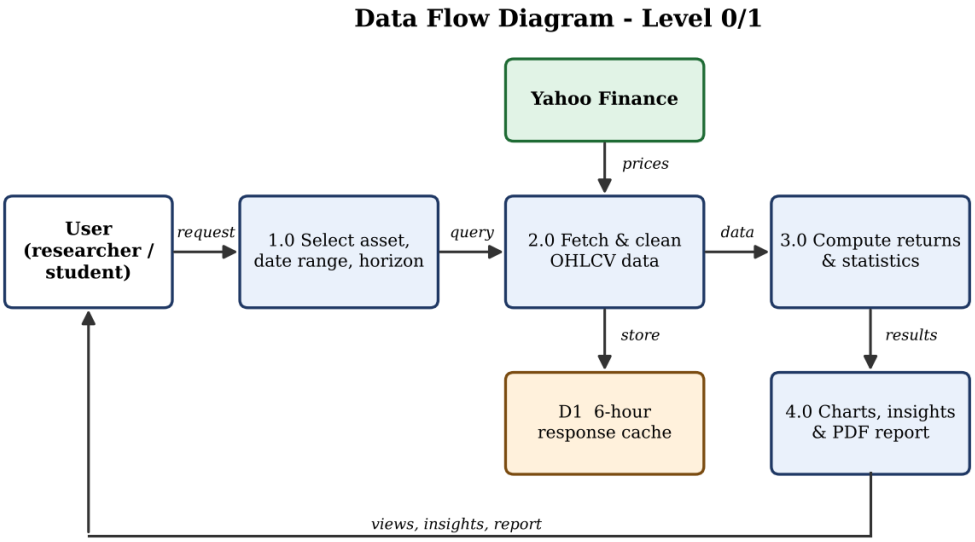


Fig No. 5.3 Data flow diagram (Level 0/1)

The principal data structures handled by the backend are summarised below.

Data set	Fields	Description
Price history (OHLCV)	Date, Open, High, Low, Close, Volume	Auto-adjusted daily history from Yahoo Finance after cleaning; weekly and monthly series are aggregated from it
Return series	Date, Return (%)	Successive close-to-close returns for the selected horizon; the first selected price has no return
Statistics summary	Mean, median, variance, SD, skewness, excess kurtosis, n	Six sample estimators per interval, not annualised
Extreme events	Date, Return, z-score, direction	Observations with |return − mean| greater than threshold × sample SD
Comparison set	Common dates, normalised prices, returns, correlation	Aligned series for two to five assets

CHAPTER 6

Implementation

6.1 Module-wise implementation

The application is organised into ten analytical modules, each available from the sidebar and each reading the same global research sample.

Module	Implementation summary
1. Overview	Price and volume chart, six statistics, density chart, direction balance and daily-return calendar.
2. Market Explorer	Candlestick, area and line charts, returns, rolling volatility, normalised and cumulative performance.
3. Distribution Lab	Adjustable density histogram, Gaussian KDE, fitted Normal curve, mean/median and sigma reference lines, Q–Q, violin, box, ECDF and tail comparisons.
4. Statistics	Six sample estimators with formula hints, standardised shape, horizon heat-map and a sortable matrix.
5. Risk Analytics	Rolling dispersion, drawdown, common-date risk/return comparison and directional risk.
6. Compare Assets	Two to five assets on a common interval: normalised prices, density overlays, correlation, spread and statistics.
7. Multi-Horizon	Daily, weekly and monthly distributions with sample sizes, shape and extreme-move frequency.
8. Extreme Events	Threshold controls, event timeline, largest returns and an inspection drawer linked to price history.
9. Insights	Calculated research findings, PDF report generation and presentation mode.
10. Data Explorer	Searchable, sortable OHLCV table with column control, pagination, source status and CSV download.

6.2 Front-end logic

The global asset, date-range and horizon controls act as one shared research sample: changing any of them updates every view. A command palette opened with **Ctrl+K** searches the asset list, and custom Yahoo symbols can be typed directly. The price chart's **Analyze visible date range** button applies the zoomed range to all statistics. The price chart and the ECharts explorer panels

synchronise their crosshairs, and in the comparison view the legend visibility is synchronised between the price and density panels. Chart PNG buttons appear in the chart toolbars, and the **Esc** key exits presentation mode and dialogs. Light and dark themes are switchable.

6.3 Backend logic

The back end performs the statistical work as follows:

- **Returns:** daily returns are calculated from successive observed closes as (current / previous − 1) × 100. No missing prices are forward-filled.
- **Aggregation:** weekly (Friday-ending) and monthly series use the last available close in each period, while OHLC and volume are aggregated within the period; boundary periods can be partial.
- **Sample size rule:** at least four returns are required, and horizons with insufficient data return a clear message.
- **Estimators:** variance and standard deviation use ddof = 1; skewness uses scipy.stats.skew with bias = False; excess kurtosis is the Fisher definition with bias = False. Constant samples have undefined skewness and kurtosis.
- **Density:** histograms are density-normalised to integrate to 1; the kernel density estimate uses Scott's bandwidth; Normal overlays use the sample mean and standard deviation.
- **Extreme events:** an observation is extreme when abs(return − mean) > threshold × sample SD; the two-sided Normal benchmark is 2 × norm.sf(threshold).
- **Risk:** drawdown is measured from the maximum prior observed close in the selected interval; rolling standard deviation uses up to 20 intervals and omits incomplete windows.

6.4 Data connectivity and caching

In place of database connectivity, the backend connects to Yahoo Finance through yfinance. Responses are cached for six hours to avoid unnecessary downloads and to make navigation between views instantaneous; pressing Refresh forces a new download. After download, invalid OHLC rows and

duplicate dates are removed and missing volume is zero-filled. The source name and retrieval time are always shown to the user.

6.5 API integration

The front end communicates with the FastAPI backend over HTTP using a local **/api** proxy, so the browser never needs to contact the backend port directly. Requests carry the asset symbol, date range and horizon; responses carry the computed series and statistics. Cross-asset comparisons align closing-price endpoints before calculating returns: weekly and monthly comparisons align common period labels, and daily stock/crypto comparisons use common observed dates, so a Friday-to-Monday crypto return spans the weekend. Assets in different currencies are compared only through percentage returns or normalised indices, never nominal prices.

6.6 Security and privacy considerations

Authentication and authorisation are not required because Folio is a single-user, local application with no accounts or stored personal data. Security is instead provided by design: the API binds only to the loopback interface on port 8000, the front end reaches it through a local proxy, fonts and libraries are bundled locally, and the application implements no trading, order or recommendation functionality.

CHAPTER 7

Features

7.1 List of main features

- Historical adjusted data from Yahoo Finance with searchable assets (Ctrl+K) and custom symbols.
- Daily, weekly and monthly return horizons with a global date range.
- Six descriptive statistics with formula hints and Normal benchmarks.
- Histogram, kernel density, fitted Normal, Q–Q, violin, box and ECDF visualisations.
- Risk analytics: rolling volatility and drawdown.
- Comparison of two to five assets with correlation and spread.
- Extreme-event detection with adjustable threshold and a Normal benchmark.
- Calculated insights, PDF research report and presentation mode.
- Data Explorer with search, sort, pagination and CSV download; chart PNG export.
- Light and dark themes; keyboard shortcuts; fully local operation.

7.2 How each feature works

Feature	User perspective	Technical description
Asset selection	Press Ctrl+K, search a company or index, or type a Yahoo symbol.	Symbol is sent to the backend, which downloads and caches adjusted history through yfinance.
Return horizon	Choose Daily, Weekly or Monthly in the control bar.	Prices are resampled using the last close of each period; returns are computed on the resampled closes.
Statistics cards	Read mean, median, SD, variance, skewness and excess kurtosis with benchmark notes.	pandas / SciPy compute sample estimators (ddof = 1; bias-corrected shape measures).

Feature	User perspective	Technical description
Distribution Lab	Adjust bins and overlays; compare the histogram with KDE and Normal fit.	Density histogram, Gaussian KDE with Scott bandwidth and a Normal curve from sample mean and SD.
Risk Analytics	View rolling volatility and drawdown.	Rolling SD over up to 20 intervals; drawdown from the maximum prior observed close.
Compare Assets	Select two to five assets and view them on a common interval.	Closing-price endpoints are aligned before returns and correlations are calculated.
Extreme Events	Move the threshold slider and inspect events in the timeline.	Events satisfy abs(return − mean) > threshold × SD; benchmark is 2 × norm.sf(threshold).
Insights and report	Read the findings, download the PDF or present them.	Descriptive findings are generated only from the calculated returns; no causes are inferred.
Data Explorer	Search, sort and export the OHLCV data.	Table is built from the cleaned dataset; the same data can be downloaded as CSV.

CHAPTER 8

Testing

8.1 Statistical (unit) testing

The correctness of the numerical engine is checked with the project's Python test script, **backend/test_stats.py**, which is run with the command python backend/test_stats.py. It validates the statistical routines – returns, variance and standard deviation, skewness, excess kurtosis and the related calculations – so that the figures shown in the interface follow the documented methodology.

8.2 Build and integration verification

The front end is verified by running **npm run build**, which type-checks the TypeScript code and produces a production build; a successful build confirms that the components, types and imports are consistent. Integration is checked by starting both services with the launcher and confirming that the front end loads data through the /api proxy, that every view renders for the selected asset, and that changing the asset, date range or horizon updates all views.

8.3 Tools used for testing

- Python test script (backend/test_stats.py) for the statistics engine.
- npm run build for TypeScript compilation and production-build verification.
- Browser-based manual testing of the ten views, using the interface itself.

8.4 Test cases and results

No.	Test case	Expected result	Observed / basis
1	Load NIFTY 50 (^NSEI), daily, 06-11-2024 to 06-11-2026	Price chart and statistical lens are displayed with source "Yahoo Finance"	Overview page loaded with 494 observations (Fig 5.1)

No.	Test case	Expected result	Observed / basis
2	View the statistics cards for a 500-observation sample	Mean, median, SD, variance, skewness and excess kurtosis are reported	Mean 0.11 %, median 0.12 %, SD 1.83 %, variance 3.335 %², skewness 0.63, excess kurtosis 11.94 (Fig 5.2)
3	Check the direction balance of the same sample	Positive + negative + unchanged equals the number of observations	268 + 231 + 1 = 500; 53.6 % positive (Fig 5.2)
4	Compare the histogram with the Normal fit	Heavy tails are flagged when kurtosis is high	Observation “Heavier tails than a Normal distribution” is shown (Fig 5.2)
5	Select fewer than four returns	Clear message that the horizon has insufficient data	Defined behaviour of the engine
6	Constant return sample	Skewness and kurtosis reported as undefined	Defined behaviour of the engine
7	Compare daily returns of a stock and a crypto-currency	Common observed dates are used; weekend return is spanned	Defined behaviour of the engine
8	Run npm run build	Type-check and production build complete without errors	Validation step documented for the project

CHAPTER 9

Deployment

9.1 Steps to deploy

On the development computer the dependencies are already installed, so the application can be started in one step:

- Double-click **Start Folio.cmd**. The launcher starts both services, checks readiness and opens the application in the browser. If Folio is already running, the existing instance is reused.
- Use the application at http://127.0.0.1:5181.
- Double-click **Stop Folio.cmd** to stop both services.

The services can also be started manually: run powershell -ExecutionPolicy Bypass -File .\start.ps1, or use two terminals – python backend/run.py for the API, and npm.cmd run dev -- --port 5181 --strictPort for the front end. Script-started services are stopped with stop.ps1; terminal-started services are stopped with Ctrl+C.

9.2 Environment configuration

For a fresh machine the following set-up is required:

- Install Python 3.11 or later and Node.js 20 or later.
- Create a Python virtual environment and install the packages from backend/requirements.txt.
- Run npm.cmd ci in the project folder to install the front-end packages.
- Optionally use the project-local backend/.packages directory (yfinance); run.py discovers it without changing global Python packages.

CHAPTER 10

Challenges & Limitations

10.1 Issues faced during development

- Financial data are imperfect: duplicate dates, invalid OHLC rows and missing volumes appear in downloaded histories.
- Different markets have different holidays, and stocks and crypto-currencies trade on different calendars, so naive alignment produces misleading comparisons.
- Weekly and monthly aggregation produces partial periods at the boundaries of a selected range.
- Assets are quoted in different currencies, so nominal prices cannot be compared directly.
- Statistical measures must follow a consistent definition to be trusted (sample versus population estimators, biased versus unbiased skewness and kurtosis).

10.2 Solutions applied

- Invalid OHLC rows and duplicate dates are cleaned, missing volume is zero-filled and weekends/holidays are not treated as missing quotes.
- Closing-price endpoints are aligned before returns are calculated; weekly and monthly comparisons align period labels and daily comparisons use common observed dates.
- Periods are labelled by their last observed date and boundary periods are documented as potentially partial.
- Comparisons use percentage returns or normalised indices rather than nominal prices.
- Sample estimators with ddof = 1 and bias-corrected skewness and excess kurtosis are used throughout and are checked by backend/test_stats.py.

10.3 Current limitations or known bugs

- Data are historical adjusted values from Yahoo Finance, not live quotes, and depend on Yahoo Finance being reachable; responses are cached for six hours.
- All return statistics are per interval and are not annualised.
- At least four returns are needed for statistics; short samples show a message instead of results.
- The application is descriptive only: it makes no predictions and gives no trading or buy/sell recommendations, and it does not infer the causes of past events.
- The launcher scripts are written for Windows.

CHAPTER 11

Future Enhancements

11.1 Planned features

The following enhancements are proposed as future work:

- Annualised return and volatility measures alongside the per-interval statistics.
- Formal normality tests (for example Jarque–Bera or Shapiro–Wilk) to complement the visual comparison with the Normal curve.
- Portfolio-level analysis, for example the return distribution of a weighted basket of stocks.
- Additional risk measures such as Value at Risk and Expected Shortfall derived from the empirical distribution.
- Export of results to Excel, and saved workspaces or watch-lists.

11.2 Possible integrations or optimizations

- Support for additional market-data providers alongside Yahoo Finance.
- Optional persistent storage of downloaded data to avoid repeated downloads.
- Cloud or multi-user hosting with user accounts, if the tool is to be shared across a class or team.
- Cross-platform launcher scripts for Linux and macOS.

CHAPTER 12

Conclusion

12.1 Summary of the project

The Stock Market Return Distribution Analyzer (Folio) studies how stock market returns behave over a selected period. It retrieves adjusted historical prices from Yahoo Finance, computes daily, weekly and monthly returns, and analyses them with the mean, median, variance, standard deviation, skewness and kurtosis, supported by histograms, density curves, Q–Q plots and risk and comparison views. The workspace is built with React and TypeScript on the front end and a FastAPI, pandas, NumPy and SciPy engine on the back end, and it runs entirely on a local computer.

12.2 What was achieved

- A working ten-view analytical workspace for return-distribution analysis, with interactive charts and light/dark themes.
- A documented and tested statistical methodology that treats missing data, partial periods, calendars and currencies carefully.
- Comparison of multiple assets, multi-horizon analysis and extreme-event detection that expose fat tails and asymmetry which a Normal model would hide.
- Automatically generated, descriptive research findings, a PDF report and a presentation mode.
- A reproducible, easy-to-launch application that fulfils the objective of helping users understand the relationship between risk and return.

12.3 Skills learned during development

- Data cleaning and time-series handling with pandas and NumPy.
- Descriptive statistics, distribution analysis, kernel density estimation and the interpretation of skewness and kurtosis with SciPy.

- Building REST APIs with FastAPI and connecting them to a React and TypeScript interface.
- Interactive data visualisation with Lightweight Charts, Plotly.js and Apache ECharts.
- Software testing, documentation and packaging a project so that it can be run reproducibly.

CHAPTER 13

References

- Yahoo Finance (historical market data source) – https://finance.yahoo.com
- yfinance library – https://pypi.org/project/yfinance/
- pandas documentation – https://pandas.pydata.org/docs/
- NumPy documentation – https://numpy.org/doc/
- SciPy statistics module (scipy.stats) – https://docs.scipy.org/doc/scipy/reference/stats.html
- FastAPI documentation – https://fastapi.tiangolo.com/
- React documentation – https://react.dev/
- TypeScript documentation – https://www.typescriptlang.org/docs/
- Tailwind CSS documentation – https://tailwindcss.com/docs
- Radix UI – https://www.radix-ui.com/
- TradingView Lightweight Charts – https://tradingview.github.io/lightweight-charts/
- Plotly.js documentation – https://plotly.com/javascript/
- Apache ECharts documentation – https://echarts.apache.org/